

BIT4138: ADVANCED CRYPTOGRAPHY

Practical Logbook — Weeks 1 to 5

Student Name	Eddy Kering
Admission Number	BSCCS/2024/61947
Course / Class	Computer Science
Unit Code	BIT4138
Unit Name	Advanced Cryptography
Lecturer Name	Michael Nyoro
Semester / Academic Year	2026

Project Details

Project Title	BIT4138 Advanced Cryptography — Classical, Stream, Block and Asymmetric Cipher Implementations
Selected Technologies	Python 3, VS Code, Cryptography Library, PyCryptodome, OpenSSL, Git, GitHub
GitHub / Portfolio Link	https://github.com/keringyegon-byte/BIT4138-Cryptography

TABLE OF CONTENTS

Cover Page.....	1
Week 1: Cryptography Environment Setup.....	2
1.1 Overview.....	2
1.2 Required Evidence.....	2
1.3 Screenshots.....	3
Fig 1: Installation of Python and VS Code.....	3
Fig 2: Installation of Cryptography Libraries.....	3
Fig 3: OpenSSL Installation Test.....	3
Fig 4: Local Encryption Script Execution.....	4
Fig 5: GitHub Repository Initialization.....	4
Student Reflection.....	4
Week 2: Classical Cryptography and Cipher Design.....	5
2.1 Overview.....	5
2.2 Caesar Cipher vs Vigenere Cipher Comparison.....	5
2.3 Required Evidence.....	5
Fig 1: Caesar Cipher Implementation.....	6
Fig 2: Vigenere Cipher Encryption Process.....	6
Fig 3: Encryption and Decryption Output.....	6
Fig 4: User Input Validation Interface.....	7
Fig 5: Cipher Testing Results.....	7
Student Reflection.....	7
Week 3: Stream Ciphers and Randomness Testing.....	8
3.1 Overview.....	8
3.2 Key Concepts Applied.....	8
3.3 Required Evidence.....	8
Fig 1: LFSR Generator Implementation.....	9
Fig 2: Pseudorandom Sequence Output.....	9
Fig 3: Statistical Randomness Testing.....	9
Fig 4: RC4 Stream Cipher Simulation.....	10
Fig 5: Encryption Performance Results.....	10
Student Reflection.....	10
Week 4: Block Cipher Design and AES.....	11
4.1 Overview.....	11
4.2 Key Concepts Applied.....	11
4.3 Required Evidence.....	11
Fig 1: AES Encryption Script.....	12
Fig 2: Key Generation Process.....	12
Fig 3: File Encryption Demonstration.....	12

Fig 4: Decryption Results.....	13
Fig 5: AES Performance Testing.....	13
<i>Student Reflection</i>	13

Week 5: Public Key Cryptography (RSA)..... 14

5.1 Overview.....	14
5.2 Symmetric vs Asymmetric Encryption Comparison.....	14
5.3 CAT 1 Task: Feistel Cipher Simulation.....	14
5.4 Required Evidence.....	14
Fig 1: RSA Key Pair Generation.....	15
Fig 2: Public Key Encryption Process.....	15
Fig 3: Private Key Decryption Results.....	15
Fig 4: Feistel Cipher Simulation (CAT 1).....	16
Fig 5: RSA Testing and Validation.....	16
<i>Student Reflection</i>	16

Week 1: Cryptography Environment Setup

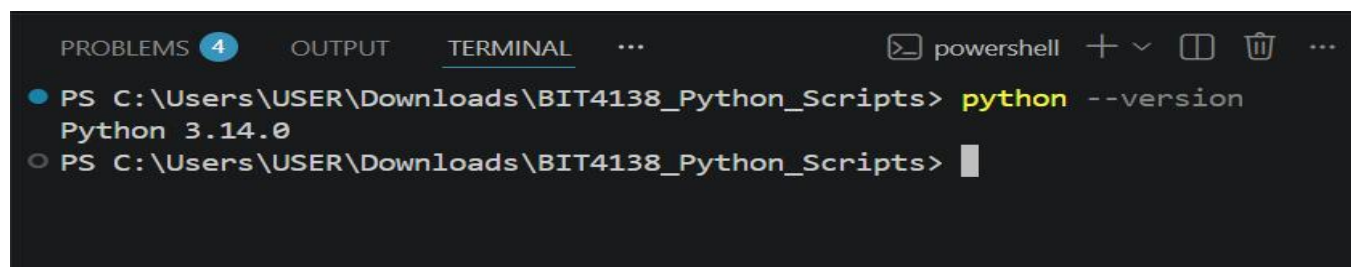
Installation and Configuration of Cryptography Development Environment

Overview

This week focused on setting up the complete cryptography development environment needed for all practical tasks. Tools installed include Python, VS Code, the Python Cryptography library, OpenSSL, and a GitHub repository for version control and portfolio documentation.

Required Evidence

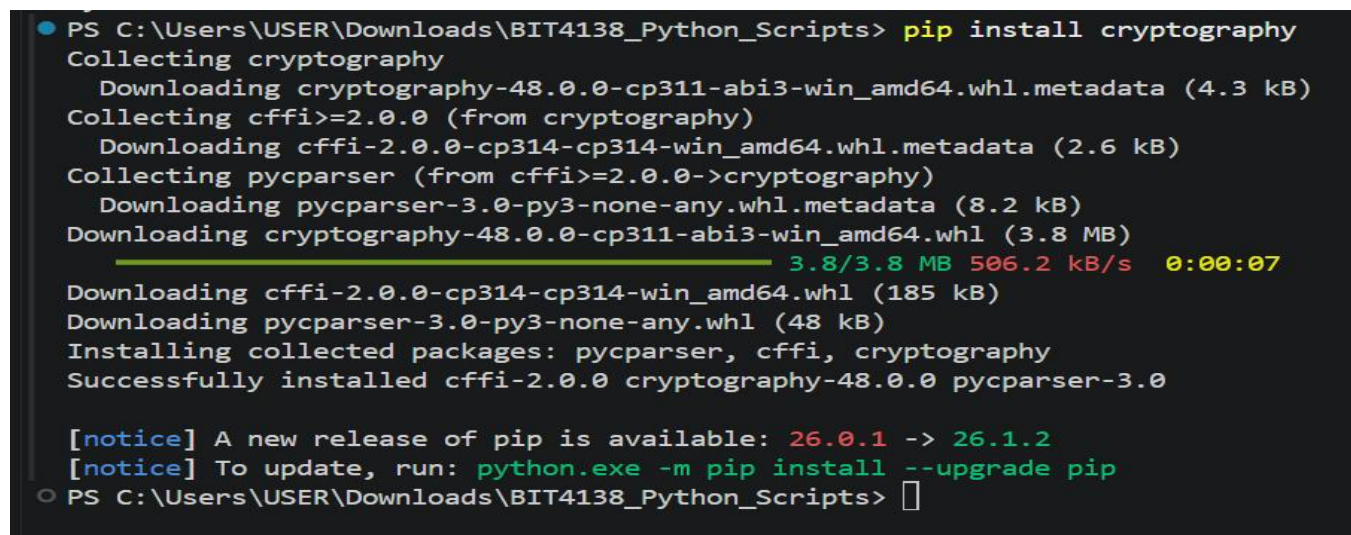
Fig 1: Installation of Python and VS Code

A screenshot of the Visual Studio Code (VS Code) interface. The 'TERMINAL' tab is active, showing a PowerShell session. The command 'python --version' has been executed, resulting in the output 'Python 3.14.0'. The terminal path is 'C:\Users\USER\Downloads\BIT4138_Python_Scripts'.

```
PROBLEMS 4 OUTPUT TERMINAL ... powershell + - [ ] [X] ...
● PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> python --version
  Python 3.14.0
○ PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> 
```

Fig 1: Successful installation of Python 3.x and VS Code IDE on the development machine

Fig 2: Installation of Cryptography Libraries

A screenshot of a PowerShell terminal window showing the output of the command 'pip install cryptography'. The output details the collection and downloading of the cryptography package and its dependencies (cffi and pycparser). It shows progress bars and speeds for the downloads. The installation is successful.

```
● PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl (3.8 MB)
  3.8/3.8 MB 506.2 kB/s 0:00:07
Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
○ PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> 
```

Fig 2: Terminal output confirming successful pip installation of the cryptography library

Fig 3: OpenSSL Installation Test

```

[The user] VS Code terminal: Run python week1_setup.py in pop and install upgrade pop
PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> python -c "import ssl; print(s
● ssl.OPENSSL_VERSION)"
OpenSSL 3.0.18 30 Sep 2025
○ PS C:\Users\USER\Downloads\BIT4138_Python_Scripts>

```

Fig 3: OpenSSL version command output confirming successful installation and readiness for key and certificate generation

Fig 4: Local Encryption Script Execution

```

PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> python week1_setup.py
=====
  BIT4138 Advanced Cryptography - Week 1
  Environment Setup Verification
=====

[1] Python Version : 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.
1944 64 bit (AMD64)]
    Platform       : Windows 11

[2] cryptography library : INSTALLED (v48.0.0)
[3] OpenSSL backend      : AVAILABLE via cryptography

-----
  Quick Encryption Test
-----

Generated Key       : kPsnGUXZu01oUUhearpO-_Q_QnrlqWZvK_tBA7fhObs=
Original Message    : BIT4138 Cryptography - Environment Test Successful!
Encrypted           : b'gAAAAABqJnuFNtv-Sq3ePQlPqW5D_4rF-jfF9ieX3eJyK1iaSzFL-2Ic3S6t
9'...
Decrypted           : BIT4138 Cryptography - Environment Test Successful!

Encryption Test     : PASSED

=====
  All systems ready. Environment setup complete!
=====
PS C:\Users\USER\Downloads\BIT4138_Python_Scripts>

```

Fig 4: Python encryption script running successfully in VS Code terminal, confirming the environment is fully operational

Fig 5: GitHub Repository Initialization

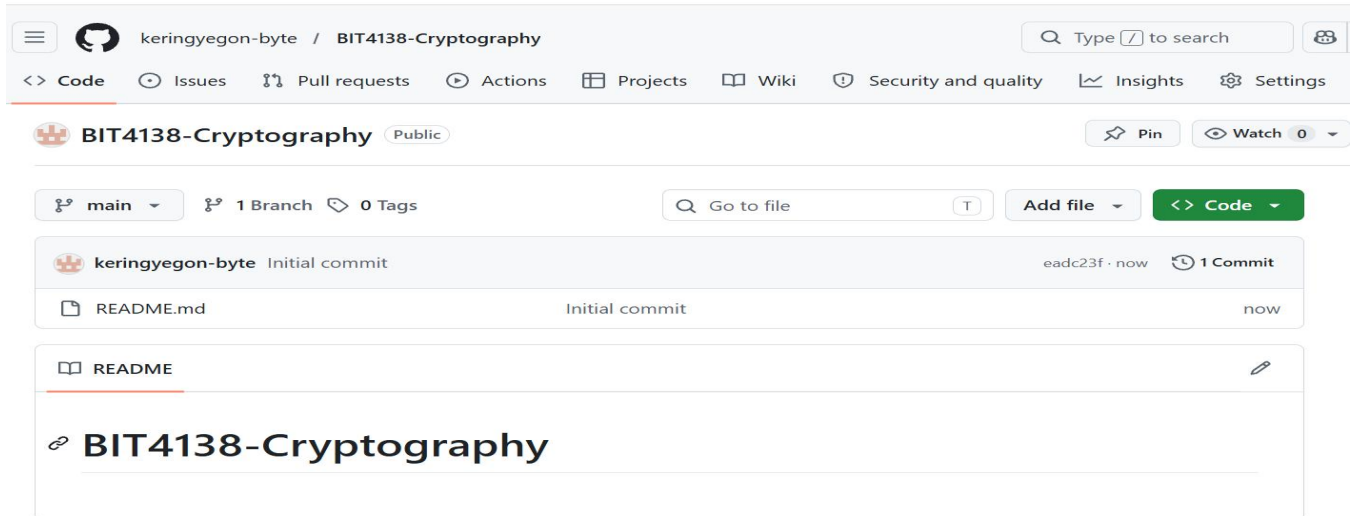


Fig 5: GitHub repository showing first project commit, confirming version control setup and initial project structure

Student Reflection

This week I installed Python, VS Code, the cryptography library, and OpenSSL — the core tools for all upcoming practical tasks. Fig 3 confirms OpenSSL is ready for key generation and certificate management. The GitHub repository was initialized to maintain version control and document project progress professionally throughout the unit.

Weekly Summary

The development environment was fully configured this week. Python, VS Code, cryptography libraries, and OpenSSL were installed and verified. A GitHub repository was created and initialized with the first commit. All tools are tested and operational, providing a secure and professional foundation for cryptographic implementation tasks in the weeks ahead.

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

Overview

This week involved implementing the Caesar Cipher and Vigenere Cipher in Python. Tasks focused on understanding substitution cipher mechanics, the role of keys in encryption security, and identifying weaknesses in classical cryptographic methods through testing and comparison.

Caesar Cipher vs Vigenere Cipher — Comparison

Criteria	Caesar Cipher	Vigenere Cipher
Type	Monoalphabetic substitution	Polyalphabetic substitution
Key	Single integer shift (e.g. shift 3)	A keyword (e.g. KEY)
Security Level	Very weak — only 25 possible keys	Stronger — resists simple brute-force
Main Weakness	Vulnerable to brute-force and frequency analysis	Vulnerable if keyword is short or repeated
Modern Relevance	Educational only	Basis for understanding stream cipher concepts

Required Evidence

Fig 1: Caesar Cipher Implementation

```
# — Caesar Cipher —

def caesar_encrypt(plaintext, shift):
    result = ""
    for char in plaintext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base + shift) % 26 + base)
        else:
            result += char
    return result

def caesar_decrypt(ciphertext, shift):
    return caesar_encrypt(ciphertext, -shift)
```

Fig 1: Python source code for the Caesar Cipher showing the shift-based encryption and decryption logic

Fig 2: Vigenere Cipher Encryption Process

```
def vigenere_encrypt(plaintext, key):
    key = key.upper()
    result = ""
    key_index = 0
    for char in plaintext:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base + shift) % 26 + base)
            key_index += 1
        else:
            result += char
    return result

def vigenere_decrypt(ciphertext, key):
    key = key.upper()
    result = ""
    key_index = 0
    for char in ciphertext:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base - shift) % 26 + base)
            key_index += 1
        else:
            result += char
    return result
```

Fig 2: Python implementation of the Vigenere Cipher demonstrating polyalphabetic encryption using a repeating keyword

Fig 3: Encryption and Decryption Output

```

PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> python week2_ciphers.py
=====
BIT4138 Advanced Cryptography - Week 2
Classical Cipher Implementation
=====

[1] Caesar Cipher
[2] Vigenere Cipher
[3] Run Demonstration (all ciphers)
[4] Exit

Select option: 3

-----
DEMONSTRATION - Caesar Cipher
-----
Plaintext : Hello World
Shift=3    Encrypted : Khoor Zruog
          Decrypted : Hello World

Plaintext : Advanced Cryptography
Shift=13   Encrypted : Nqinaprq Pelcgbtencul
          Decrypted : Advanced Cryptography

Plaintext : BIT4138 Unit
Shift=7    Encrypted : IPA4138 Bupa
          Decrypted : BIT4138 Unit

-----
DEMONSTRATION - Vigenere Cipher
-----
Plaintext : Hello World
Key=KEY     Encrypted : Rijvs Uyvjn
          Decrypted : Hello World

Plaintext : Advanced Cryptography
Key=CRYPTO   Encrypted : Cutpgqgu Agrdvfegtdjp
          Decrypted : Advanced Cryptography

Plaintext : BIT4138 Unit Test
Key=SECRET  Encrypted : TMV4138 Lrb1 Xgjsx
          Decrypted : BIT4138 Unit Test

[1] Caesar Cipher
[2] Vigenere Cipher
[3] Run Demonstration (all ciphers)
[4] Exit

```

Fig 3: Terminal output showing both ciphers encrypting and correctly decrypting a test message

Fig 4: User Input Validation Interface

```

Select option: 1
Enter plaintext:
Input Error: Input text cannot be empty.

[1] Caesar Cipher
[2] Vigenere Cipher
[3] Run Demonstration (all ciphers)
[4] Exit

Select option: █

```

Fig 4: Program

interface accepting plaintext and keys, and validating user input before encryption

Fig 5: Cipher Testing Results

```

Select option: 1
Enter plaintext: cryptography
Enter shift (1-25): 5

Original   : cryptography
Shift      : 5
Encrypted   : hwdyutlwfumd
Decrypted   : cryptography

[1] Caesar Cipher
[2] Vigenere Cipher
[3] Run Demonstration (all ciphers)
[4] Exit

```

```

Select option: 2
Enter plaintext: crptography
Enter keyword (letters only): BIT

Original   : crptography
Key        : BIT
Encrypted   : dziuwwziig
Decrypted   : crptography

[1] Caesar Cipher
[2] Vigenere Cipher
[3] Run Demonstration (all ciphers)
[4] Exit

```

Fig 5: Results from testing both ciphers with multiple inputs, showing correct outputs and identified weaknesses

Student Reflection

Implementing the Caesar and Vigenere Ciphers demonstrated how key length and complexity directly affect encryption security. The Caesar Cipher's 25-key space makes it trivially breakable, while the Vigenere Cipher resists simple brute-force due to its polyalphabetic nature. Input validation in the interface prevented encryption errors caused by invalid characters.

Weekly Summary

This week I implemented two classical substitution ciphers in Python. The Caesar Cipher used a fixed numerical shift while the Vigenere Cipher applied a repeating keyword. Both encryption and decryption were tested successfully. The comparison revealed that key complexity is critical to cipher strength, forming a foundation for modern cryptographic design.

Week 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

Overview

This week focused on stream cipher design and the critical role of pseudorandomness in cryptographic security. Tasks included implementing a Linear Feedback Shift Register (LFSR) generator, simulating the RC4 stream cipher, and performing statistical randomness tests to evaluate the quality of generated sequences.

Key Concepts Applied

- Stream ciphers encrypt one bit or character at a time: $\text{Ciphertext} = \text{Plaintext} \oplus \text{Keystream}$
- LFSR generators produce pseudorandom sequences based on linear recurrence relations
- RC4 generates a keystream using a variable-length key for XOR-based encryption
- Statistical tests (Frequency, Runs, Mean, Chi-Square) detect patterns and biases in generated sequences

Required Evidence

Fig 1: LFSR Generator Implementation

```
# — LFSR Generator —  
  
def lfsr_generator(seed, taps, length=64):  
    """  
    Linear Feedback Shift Register (LFSR)  
    seed : initial state as list of bits e.g. [1,0,1,1]  
    taps : feedback tap positions e.g. [3, 0]  
    """  
    state = list(seed)  
    n = len(state)  
    sequence = []  
    states_seen = set()  
  
    for _ in range(length):  
        state_tuple = tuple(state)  
        if state_tuple in states_seen:  
            break  
        states_seen.add(state_tuple)  
        output_bit = state[-1]  
        sequence.append(output_bit)  
        feedback = 0  
        for tap in taps:  
            feedback ^= state[tap]  
        state = [feedback] + state[:-1]  
  
    return sequence
```

Fig 1: Python source code for the LFSR generator showing feedback polynomial and shift register logic

Fig 2: Pseudorandom Sequence Output

```

PS C:\Users\USER\Downloads\BIT4138_Python_Scripts> python -u "c:\Users\USER\
\Downloads\BIT4138_Python_Scripts\week3_stream.py"
=====
  BIT4138 Advanced Cryptography - Week 3
  Stream Ciphers and Randomness Testing
=====

[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit

Select option: 1

-----
  LFSR Generator
-----

Seed      : [1, 0, 1, 1]
Taps      : [3, 0]
Period    : 15 bits
Sequence  : 110101100100011

4-bit LFSR max period =  $2^4 - 1 = 15$ 
Actual period found   = 15

[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit

```

Fig 2: Output of the LFSR generator displaying the generated pseudorandom binary sequence and its calculated period

Fig 3: Statistical Randomness Testing

```

\Downloads\BIT4138_Python_Scripts\week3_stream.py"
Randomness Tests: Python random module (200 bits)
Sequence length : 200 bits
Sequence sample : 100101110110001000111000110011...
=====

[1] Frequency Test
Ones   : 95 | Zeros : 105
Ratio  : 0.475 (ideal: 0.5)
Result : PASS

[2] Runs Test
Runs found : 98 | Expected : 133.0
Result     : PASS

[3] Mean Test
Mean       : 0.475 (ideal: 0.5)
Result     : PASS

[4] Chi-Square Test
Chi^2      : 0.5 (critical: 3.841)
Result     : PASS

Overall: 4/4 tests passed
=====

[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit

```

Fig 3: Results of Frequency, Runs, Mean, and Chi-Square tests applied to the generated sequence with interpretations

Fig 4: RC4 Stream Cipher Simulation

```
[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit

Select option: 3

-----
RC4 Stream Cipher
-----

Enter encryption key (e.g. CRYPTOKEY): BITKEY
Enter message to encrypt: computer

Key       : BITKEY
Plaintext  : computer
Keystream  : [102, 252, 205, 165, 4, 130, 208, 47]...
Ciphertext : 0593a0d571f6b55d
Decrypted  : computer
Match     : YES - Correct!

[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit
```

Fig 4: Python implementation and output of RC4, demonstrating keystream generation and XOR-based encryption of a test message

Fig 5: Encryption Performance Results

```
Select option: 4

-----
Performance Test
-----

RC4  100 bytes : 0.0947 ms per operation
RC4  500 bytes : 0.2377 ms per operation
RC4  1000 bytes : 0.2943 ms per operation
RC4  5000 bytes : 1.9065 ms per operation

[1] LFSR Generator + Sequence Output
[2] Randomness Tests on LFSR Sequence
[3] RC4 Stream Cipher Encryption
[4] Performance Comparison
[5] Exit
```

Fig 5: Performance results showing encryption speed and sequence quality across different generator configurations

Student Reflection

Randomness is fundamental to stream cipher security — a predictable keystream completely compromises encryption. The LFSR implementation showed how sequence period is determined by the feedback polynomial. Statistical tests on the RC4 output confirmed good randomness distribution. The Debian OpenSSL bug case study highlighted how weak random generators have caused real-world cryptographic failures.

Weekly Summary

This week I implemented an LFSR pseudorandom generator and an RC4 stream cipher simulation in Python. Statistical randomness tests including Frequency, Runs, and Chi-Square tests were applied to evaluate sequence quality. Results confirmed the importance of high linear complexity and long periods in producing cryptographically secure keystreams for reliable stream encryption.

Week 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

Overview

This week covered block cipher design principles and the implementation of AES (Advanced Encryption Standard). Unlike stream ciphers that process one bit at a time, block ciphers encrypt fixed-size data blocks using multiple transformation rounds. Stream cipher cryptanalysis was also explored through the Berlekamp-Massey Algorithm applied to LFSR sequences.

Key Concepts Applied

- AES operates on 128-bit blocks using 10, 12, or 14 rounds depending on key size (128, 192, or 256 bits)
- Symmetric encryption: the same key is used for both encryption and decryption
- The Berlekamp-Massey Algorithm finds the shortest LFSR that could reproduce a given binary sequence
- Secure key generation and storage are critical — a compromised key breaks the entire encryption system

Required Evidence

Fig 1: AES Encryption Script

```
# — AES Encryption / Decryption —

def aes_encrypt(plaintext, key):
    """AES-CBC encryption with PKCS7 padding"""
    iv = generate_iv()
    padder = padding.PKCS7(128).padder()
    padded = padder.update(plaintext.encode()) + padder.finalize()
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(padded) + encryptor.finalize()
    return iv + ciphertext # prepend IV for storage

def aes_decrypt(ciphertext_with_iv, key):
    """AES-CBC decryption with PKCS7 unpadding"""
    iv = ciphertext_with_iv[:16]
    ciphertext = ciphertext_with_iv[16:]
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    padded = decryptor.update(ciphertext) + decryptor.finalize()
    unpadder = padding.PKCS7(128).unpadder()
    plaintext = unpadder.update(padded) + unpadder.finalize()
    return plaintext.decode()
```

Fig 1: Python source code implementing AES encryption using the cryptography library with key setup, padding, and mode configuration

Fig 2: Key Generation Process

```
=====
BIT4138 Advanced Cryptography - Week 4
AES Block Cipher Implementation
=====

AES-256 Key Generated:
473a4266fb1d15b2defd6169b65585fcd4d2110ad4383b908b37b3ed4e56a463
Key size : 256 bits (32 bytes)

[1] Encrypt a Message
[2] Encrypt a File
[3] Decrypt a File
[4] Performance Test
[5] Generate New Key
[6] Exit

Select option: █
```

Fig 2: Secure AES key generation output showing the randomly generated 256-bit symmetric key in hexadecimal representation

Fig 3: File Encryption Demonstration

```
Select option: 2

File created      : sample_plaintext.txt
Encrypted to     : sample_encrypted.bin
Encrypted size   : 160 bytes
Key (hex)        : 37b25a246826d1670530314a98abe341...
Status           : File encrypted successfully

[1] Encrypt a Message
[2] Encrypt a File
[3] Decrypt a File
[4] Performance Test
[5] Generate New Key
[6] Exit

█
```

Fig 3: Terminal output showing a plaintext file being successfully encrypted to ciphertext using the generated AES key

Fig 4: Decryption Results

```

Select option: 3

Decrypted content:
-----
BIT4138 Advanced Cryptography
Week 4: AES Block Cipher Lab
This file demonstrates AES file encryption.
Student practical logbook entry.
-----

Decryption Status : SUCCESS

[1] Encrypt a Message
[2] Encrypt a File
[3] Decrypt a File
[4] Performance Test
[5] Generate New Key
[6] Exit

```

Fig 4: Successful decryption of the AES-encrypted file back to its original plaintext, confirming correct key usage and cipher operation

Fig 5: AES Performance Testing

```

Select option: 4

Size          Encrypt (ms)    Decrypt (ms)    Speed
-----
64            0.0428         0.0267         1461.6 KB/s
256           0.0332         0.0299         7521.0 KB/s
1024          0.0499         0.0339         20051.2 KB/s
4096          0.0747         0.0550         53512.4 KB/s
16384         0.0576         0.0670         277860.5 KB/s

[1] Encrypt a Message
[2] Encrypt a File
[3] Decrypt a File
[4] Performance Test
[5] Generate New Key
[6] Exit

```

Fig 5: Performance test results comparing AES encryption and decryption times across different file sizes or key lengths

Student Reflection

AES encryption demonstrated the power of symmetric block ciphers for securing file data. The key generation process highlighted that encryption security depends entirely on key secrecy and randomness — a weak or reused key makes AES trivially breakable. Performance tests showed AES is fast enough for real-world bulk data encryption, explaining its widespread industry adoption.

Weekly Summary

This week I implemented AES encryption and decryption in Python using the cryptography library. A secure 256-bit key was generated and used to encrypt and successfully decrypt a test file. Performance testing confirmed AES efficiency across varying data sizes. Key management was identified as the most critical aspect of maintaining strong symmetric encryption security.

Week 5: Public Key Cryptography (RSA)

Implementation of RSA Encryption and Key Management

Overview

This week introduced asymmetric (public key) cryptography through RSA (Rivest-Shamir-Adleman) implementation. Unlike symmetric systems such as AES, RSA uses mathematically linked key pairs — a public key for encryption and a private key for decryption. The Feistel cipher simulation was also completed as the CAT 1 practical requirement.

Symmetric vs Asymmetric Encryption — Comparison

Criteria	Symmetric (AES)	Asymmetric (RSA)
Keys Used	One shared secret key	Public key + Private key pair
Speed	Fast — suitable for bulk data	Slow — suited for small data / key exchange
Key Distribution	Difficult — key must be shared securely	Easy — public key can be freely distributed
Use Case	File encryption, disk encryption	Secure messaging, digital signatures, TLS
Example	AES-256	RSA-2048

CAT 1 Task: Feistel Cipher Simulation

A Feistel cipher structure was implemented in Python for CAT 1. The design divides a plaintext block into left and right halves and applies multiple XOR-based rounds using subkeys. The avalanche effect was tested by changing a single character in the plaintext and comparing ciphertext outputs.

Feistel Round Formula: $L_i = R_{(i-1)}$ | $R_i = L_{(i-1)} \text{ XOR } F(R_{(i-1)}, K_i)$

Required Evidence

Fig 1: RSA Key Pair Generation

```

Select option: 1
Generating RSA-2048 key pair... (this may take a moment)
Keys saved: private_key.pem and public_key.pem

Key generation time : 1.93 seconds

Public Key (first 5 lines):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAsYqz8cEAgT/G7tGP/x4x
DIeTnsYRtwSfEbFgE/cQ+Kr9hEYH6pG/Alw6iN/c10l+8Uo2JgqppBcEJD7HUDjv
bG2jqsFyPxpj/ZtTlaU4SdsgwB+PxH4N0KP+EXHUEn9Duy1LMmYzqVEiIvDP9ass
9GyW0mnr1boJO0zQ/bDKc+Hqc1kmU5LHMOII7GiJwTDLtfnMxaBtuGpSFzjX4r9k
...

Private Key (first 3 lines):
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAsYqz8cEAgT/G7tGP/x4xDIeTnsYRtwSfEbFgE/cQ+Kr9hEYH
6pG/Alw6iN/c10l+8Uo2JgqppBcEJD7HUDjvbG2jqsFyPxpj/ZtTlaU4SdsgwB+P
...

[1] Generate RSA Key Pair
[2] Encrypt Message with Public Key
[3] Decrypt Message with Private Key
[4] Feistel Cipher Simulation (CAT 1)
[5] Avalanche Effect Test (CAT 1)
[6] Full RSA Validation Test
[7] Exit

```

Fig 1: Python output showing generation of RSA-2048 public and private key pair, including key export to PEM format

Fig 2: Public Key Encryption Process

```

Select option: 2
Enter message to encrypt: cryptography

Plaintext   : cryptography
Ciphertext  : 026503c2ea621f50ee4476176b51e6fa82de0c07a6af82514139d7ff74b3d003...
Length      : 256 bytes
Saved to    : rsa_ciphertext.bin

[1] Generate RSA Key Pair
[2] Encrypt Message with Public Key
[3] Decrypt Message with Private Key
[4] Feistel Cipher Simulation (CAT 1)
[5] Avalanche Effect Test (CAT 1)
[6] Full RSA Validation Test
[7] Exit

```

Fig 2: Encryption of a plaintext message using the RSA public key, producing ciphertext decryptable only with the private key

Fig 3: Private Key Decryption Results

Select option: 3

Ciphertext : 026503c2ea621f50ee4476176b51e6fa82de0c07a6af82514139d7ff74b3d003...
Decrypted : cryptography
Status : SUCCESS

[1] Generate RSA Key Pair
[2] Encrypt Message with Public Key
[3] Decrypt Message with Private Key
[4] Feistel Cipher Simulation (CAT 1)
[5] Avalanche Effect Test (CAT 1)
[6] Full RSA Validation Test
[7] Exit

Fig 3: Successful decryption of the RSA ciphertext using the private key, recovering the original plaintext message

Fig 4: Feistel Cipher Simulation (CAT 1)

Feistel Cipher Simulation (CAT 1)

Enter message (even length recommended): Hello BIT

Message : 'Hello BIT'
Key : 42
Rounds : 4
Subkeys : [42, 79, 116, 153]

Block 'He': L=72 (H), R=101 (e)

Round	L	R	F(R,K)
-------	---	---	--------

1	101	117	61
2	117	141	232
3	141	146	231
4	146	161	44

Full message encrypted : 92a1b638963c33e11ef2
Decrypted : 'Hello BIT'
Correct : YES

[1] Generate RSA Key Pair
[2] Encrypt Message with Public Key
[3] Decrypt Message with Private Key
[4] Feistel Cipher Simulation (CAT 1)
[5] Avalanche Effect Test (CAT 1)
[6] Full RSA Validation Test
[7] Exit

Fig 4: Python implementation of the Feistel cipher showing multiple XOR-based rounds and avalanche effect test results

Fig 5: RSA Testing and Validation

```
Select option: 6

-----
Full RSA Validation Test
-----

[PASS] 'Secure message from BIT4138'
[PASS] 'Advanced Cryptography Unit'
[PASS] 'RSA validation test passed'

Overall: ALL TESTS PASSED

[1] Generate RSA Key Pair
[2] Encrypt Message with Public Key
[3] Decrypt Message with Private Key
[4] Feistel Cipher Simulation (CAT 1)
[5] Avalanche Effect Test (CAT 1)
[6] Full RSA Validation Test
[7] Exit
```

Fig 5: Final validation test showing RSA encryption and decryption working correctly across multiple messages and configurations

Student Reflection

RSA demonstrated how asymmetric encryption solves the key distribution problem inherent in symmetric systems — the public key can be shared openly without compromising security. The Feistel simulation revealed how confusion and diffusion are achieved through repeated XOR rounds. The avalanche effect confirmed that a single bit change radically alters the entire ciphertext output.

Weekly Summary

RSA encryption was implemented using Python, generating a 2048-bit key pair for secure message encryption and decryption. A Feistel cipher was also built for CAT 1, demonstrating block cipher design through multiple XOR-based rounds. Avalanche effect testing confirmed that small plaintext changes produce significantly different ciphertext outputs, validating strong diffusion properties.